

Teaching Statement

Ziyi Chen

University of Maryland College Park

When I saw the high school students stare at my interesting high-tech example with amazement and curiosity in my invited speech (see Section 2.1), I could imagine them growing up to be scientists with a strong spirit of exploration. I realize that teaching is not only sharing knowledge, but also inspiring enthusiasm in exploration, which guides my teaching experience and philosophy.

1 Prior Teaching and Mentoring Experiences

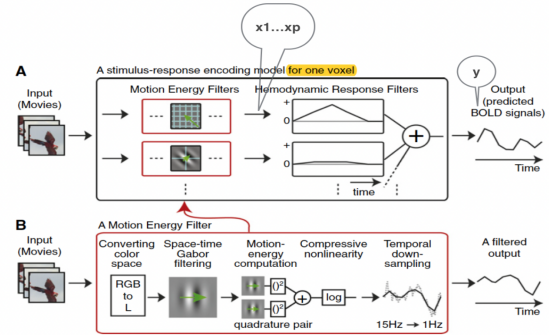
1. I have taught reinforcement learning at the University of Maryland, College Park, covering selected important applications and algorithms (see Section 2.2) in two 75-minute lectures, while the students told me that they understood clearly.
2. I have taught recitation sections of exercises on multivariate calculus at Cornell University and Peking University. In 2014 Spring, I was invited by Peking University to share my TA experiences with about 60 audiences.
3. I have been invited to give a talk titled “Statistics in real life” to students at Ithaca High School, where students were fascinated by the interesting application examples (see Section 2.1).
4. I have voluntarily tutored mathematics and programming languages like Python and R, where I helped students learn by doing with immediate feedback (see Section 2.3).
5. I have mentored students on research projects as a postdoc at the University of Maryland, College Park, with research results including an ICML 2024 paper, an Neurips 2024 paper and two ICLR 2025 papers (under review).

2 Teaching and Mentoring Philosophy

2.1 Motivate Students with Relevant, Useful and Interesting Application Examples

I usually select application examples that are **relevant**, **useful** and **interesting** to motivate students. For example, in my invited talk titled “Statistics in Real Life” to Ithaca High School students, I introduced an application of statistical regression in “mind-reading”, which can decode imaginary video in people’s minds based on the fMRI brain signal, as shown in Figure 1. This example is **relevant** because it concretely explains the concept of statistical regression, which is an important extension of linear regression that these high school students learned in their curriculum. This example is **useful** in multiple applications, including communication with paralysis patients and reproduction of dreams and memories. This example is also so **interesting** that these students stared at my slides with amazement and curiosity, and one of them even expressed her wish to learn more about this fMRI application in the future.

Core: “We built a model for each voxel that describes how **shape and motion information (X) in the movie** is mapped into **brain activity (Y)**.” —Nishimoto.



2.2 Brief and Clear Structure

A brief and clear structure is important to help students focus on important contents and understand them well. In particular, I ensure brevity by carefully selecting important contents, and ensure clarity by showing the catalog of these selected contents in a logical sequence to the audience. An example is shown in Figure 2 in my first 75-minute lecture on reinforcement learning (RL), which consists of 4 main sections in a logical sequence: RL problem (introduction to basic RL problem), policy evaluation (task 1 to basic RL problem), policy optimization (task 2 to basic RL problem which sometimes rely on task 1), and advanced RL settings (extension of basic RL problem to advanced ones). To ensure brevity, I only selected the most basic and popular algorithms (subsections in Figure 2) among the large number of existing ones for the two fundamental RL tasks, namely policy evaluation and policy optimization (sections 2-3 in Figure 2). To ensure clarity, I not only used logical sequence explained above, but also showed the this catalog slide at the beginning of each subsection, with current section and subsection titles bolded. The students told me that they understood this lecture clearly.

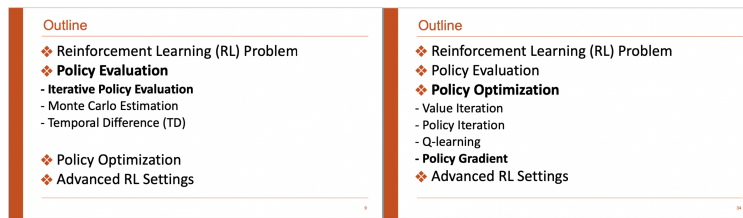


Figure 2: Two Slides on the Outline of Reinforcement Learning

```
In [1]: import numpy as np
...: x=np.array([0,1,2,3])
...: x_alias=x
...: x_copy=x.copy() #Correct
...: x[2]=2222
...: print("x:")
...: print(str(x))
...: print("\n x_alias:")
...: print(x_alias)
...: print("\n x_copy:") #Correct
...: print(x_copy)

x:
[ 0  1 2222  3]
x_alias:
[ 0  1 2222  3]
x_copy:
[0 1 2 3]
```

```
In [2]: import copy
...: y={'a':np.array([0,1,2,3])}
...: y_copy=y.copy()
...: y_copy2=copy.copy(y)
...: y_deepcopy=copy.deepcopy(y) #Correct
...: y['a'][2]=2222
...: print("y:")
...: print(str(y))
...: print("\n y_copy:")
...: print(y_copy)
...: print("\n y_copy2:")
...: print(y_copy2)
...: print("\n y_deepcopy:") #Correct
...: print(y_deepcopy)

y:
{'a': array([ 0,  1, 2222,  3])}
y_copy:
{'a': array([ 0,  1, 2222,  3])}
y_copy2:
{'a': array([ 0,  1, 2222,  3])}
y_deepcopy:
{'a': array([0, 1, 2, 3])}
```

Figure 3: Learn Python Aliasing by Doing with Immediate Feedback

2.3 Learn by Doing with Immediate Feedback

Many people love computer games because game players have the power to operate and obtain immediate feedback to motivate players to adjust their operations. This phenomenon motivates me to let students learn by doing with immediate feedback, etc. For example, when I taught Python code, I wrote Python file containing very commonly used Python commands and shared with students, so that students immediately see the output results for each command. After class, they can furthermore consolidate their memory by running and adjusting the commands they are unfamiliar with and immediately see the results. A more specific instance is shown in Figure 3, where the output results obviously illustrate how to use copy and deep copy to avoid undesired aliasing, which is easier to understand than a paragraph describing such knowledge.

Such immediate feedbacks have also been used by my advisors and me in research mentorship. For example, my postdoc advisor requires our weekly written reports, and point out if our progress or proposed research plan needs to improve. My PhD advisor holds weekly meeting where we mentees report our research progress and encountered difficulties, and he praises the great ideas and progress, points out directions for improvement, and helps brainstorm ideas to overcome the difficulties. I have enjoyed and benefited a lot from such immediate feedbacks from my PhD advisor which brings me sense of achievement and belonging, the motivation to improve, the relief from significant research difficulties, and most importantly, the research methodologies (e.g. how to find novel and significant research gap, how to debug when theory and experiments yield undesired results, the logic sequence in paper writing, etc.). As I become more experienced in research, my advisor gradually reduces the frequency of feedbacks so that I can lead research projects and improve my independent research skills. I followed my PhD advisor's style when I mentored PhD students on research projects as a postdoc at the University of Maryland, College Park. Specifically, I provide brainstorm, encouragement, guidance, etc. as constructive feedbacks when my mentees make one step forward or face difficulties. My collaborations with my mentees are also fruitful and pleasant which produce an ICML 2024 paper, a Neurips 2024 paper and two ICLR 2025 papers (under review).

3 Example Courses to Teach in the Future

I have a solid background in math, statistics, theoretical computer science and machine learning, and also have coding skills and experiences. The courses I am able to teach include, but are not limited to: introduction to machine learning, seminar courses on advanced machine learning topics (e.g. optimization theory, statistical learning theory, reinforcement learning, Markov process, deep learning, large language models, diffusion models, trustworthy AI by alignment with human value, etc.), computer algorithms and computation complexity analysis, discrete math, calculus, probability, linear algebra, probabilistic graphical models, signal processing, programming languages (e.g., Python, Julia, MATLAB, R, etc.), etc.